

PIPELINE OPERATIONS GUIDE

AIOps PLE Platform

941K Users x 12 Tasks x 7 Shared Experts

Target: ML Engineers (Operators)

2026-04-01 | Config-Driven Architecture

Table of Contents

1. System Requirements	4
1.1. Hardware	4
1.2. Software	4
1.3. AWS Credentials	4
1.4. Docker Environment	4
1.5. Deployment Environments	5
1.6. Directory Structure (Key)	5
2. Phase 0: Feature Engineering	6
2.1. Execution Flow	6
2.2. Adapter Execution	6
2.3. PipelineRunner — Preprocessing Pipeline	6
2.3.1. 3-Stage Normalization (Must Understand)	6
2.4. Generator Input Routing	7
2.5. Phase 0 Output Verification (Pre-flight Check)	7
2.6. SageMaker Processing Job (Cloud)	7
3. Phase 1–3: Training + Ablation	8
3.1. Training Architecture Overview	8
3.2. Execution Modes	8
3.2.1. Mode 1: Local Direct Execution (Development/Debugging)	8
3.2.2. Mode 2: Docker Local Execution (Reproducing SageMaker Environment)	8
3.2.3. Mode 3: SageMaker Cloud Execution	9
3.3. Ablation Execution (4-Dimension, 23 Scenarios)	9
3.3.1. Ablation Structure	9
3.3.2. Local Ablation Execution	10
3.3.3. SageMaker Ablation Execution (Parallel)	10
3.3.4. Ablation Hyperparameters	10
3.3.5. Ablation Default Training Parameters	11
4. Phase 4: Knowledge Distillation	12
4.1. Teacher -> LGBM Student	12
4.2. Execution CLI	12
4.3. Distillation Settings (pipeline.yaml)	12
4.4. Fidelity Gate	13
4.5. IG-Based Feature Selection (For Distillation)	13
5. Phase 5: Serving	14
5.1. 3-Tier Scaling Architecture	14
5.2. Lambda Deployment	14
5.3. Inference Flow	14
5.4. Recommendation Rationale Generation	15
5.5. A/B Testing	15
6. Config Guide	16
6.1. Split-Config Architecture	16
6.2. pipeline.yaml Key Settings	16

6.2.1. tasks Section	16
6.2.2. training Section	16
6.2.3. data Section	17
6.2.4. aws Section	17
6.2.5. cold_start Section	17
6.3. feature_groups.yaml Core Structure	18
6.4. task_groups Section (GradSurgery / task-type projection experiment)	18
6.5. task_relationships (Logit Transfer)	19
7. Monitoring	20
7.1. Data Drift	20
7.2. Model Performance Monitoring	20
7.3. Fairness Audit	20
7.4. Lineage Tracking	20
7.5. Champion/Challenger Evaluation (Offline Gate)	21
8. Troubleshooting (FAQ)	22
8.1. NaN Loss During Phase 2 Training	22
8.2. Low GPU Utilization (< 50%)	22
8.3. Suspected Label Leakage	22
8.4. SageMaker Job Timeout	23
8.5. Frequent Spot Instance Interruptions	23
8.6. Gradient Norm Explosion	23
9. Cost Management	25
9.1. Spot Instance Strategy	25
9.2. Cost Check CLI	25
9.3. Budget Guard	25
9.4. Cost Optimization Checklist	25
9.5. Infrastructure Cost Comparison	26
9.6. Orchestration Cost	26
10. Appendix: Operations Rules Summary	27
10.1. Absolute Prohibitions	27
10.2. Separation of Concerns Principle	27
10.3. Development Order (Must Follow)	27
10.4. Checkpoint Resume	27
10.5. Pipeline State Auto-Recovery	28
11. Ops/Audit Agent Integration	29

System Requirements

Hardware

Category	Local Development	AWS Cloud
GPU	RTX 4070 12GB (or equivalent)	g4dn.xlarge (T4 16GB)
RAM	64GB or more	ml.m5.2xlarge (32GB) — Phase 0
Storage	SSD 100GB+	S3 (unlimited)
CPU	8+ cores	Auto-provisioned

Software

```
Python 3.10+
PyTorch 2.1+ (CUDA 12.1)
DuckDB 1.0+
cuDF (optional, for GPU acceleration)
Docker 24+ (local ablation)
AWS CLI v2 + SageMaker SDK
```

AWS Credentials

```
# Configure AWS CLI profile
aws configure --profile ple-platform
# Region: ap-northeast-2
# IAM Role: AWSPLEPlatformSageMakerRole

# Verify current credentials
aws sts get-caller-identity --profile ple-platform

# Verify S3 bucket access
aws s3 ls s3://aiops-ple-financial/ --profile ple-platform
```

Docker Environment

```
# Build training container (one-time)
docker build -t ple-training:latest -f containers/training/Dockerfile .

# Test GPU access
docker run --rm --gpus all ple-training:latest nvidia-smi
```

Deployment Environments

The pipeline operates identically in two environments:

- **AWS:** SageMaker Training Job (training) + Lambda/ECS (serving) + Bedrock (reasons/agents)
- **On-premises (air-gapped):** Local GPU (RTX 4070, 64GB RAM) + Docker + vLLM (Exaone/Qwen)

Code and config are identical; environment detection (`SM_MODEL_DIR` presence) handles auto-switching.

Directory Structure (Key)

```
aws_ple_for_financial/
+-- configs/
|   +-- pipeline.yaml           # Common pipeline configuration (shared across
datasets)
|   +-- datasets/
|       +-- santander.yaml     # Dataset-specific overrides (id_col, label_cols,
paths)
|       +-- santander/
|           +-- feature_groups.yaml # 5-axis feature group definitions
+-- containers/training/
|   +-- train.py               # SageMaker training entry point
|   +-- Dockerfile
+-- adapters/
|   +-- santander_adapter.py    # raw data -> DataFrame
+-- core/
|   +-- pipeline/runner.py      # Phase 0 execution
|   +-- training/trainer.py     # PLETrainer
|   +-- config_builder.py       # Single source of truth for PLEConfig (reads
pipeline.yaml + dataset yaml)
|   +-- serving/predict.py      # Inference service
+-- scripts/
|   +-- run_local_ablation.py   # Local ablation
|   +-- run_santander_ablation.py # SageMaker ablation orchestrator
|   +-- run_sagemaker_teacher.py # SageMaker: submit teacher training job
|   +-- run_sagemaker_eval.py   # SageMaker: submit evaluation job
|   +-- run_sagemaker_distillation.py # SageMaker: submit distillation job
|   +-- package_source.py       # Build and stage source package to S3 (run once,
reuse)
+-- data/benchmark/
    +-- benchmark_v2.parquet     # 941K user data
```

Phase 0: Feature Engineering

Phase 0 transforms raw data into training-ready tensors. It runs on CPU instances (no GPU waste).

Execution Flow

```
Raw Parquet -> Adapter -> PipelineRunner -> Training-ready Artifacts
    +-- features.parquet
    +-- labels.parquet
    +-- sequences/ (3D tensors)
    +-- feature_schema.json
    +-- feature_stats.json
    +-- label_stats.json
```

Adapter Execution

The Adapter converts raw data into a standardized DataFrame. It does not perform preprocessing, feature generation, or label derivation.

```
# Local execution (DuckDB backend)
python -m core.pipeline.runner \
  --config configs/santander/pipeline.yaml \
  --stage adapter \
  --backend duckdb
```

PipelineRunner — Preprocessing Pipeline

```
# Full Phase 0 execution
python -m core.pipeline.runner \
  --config configs/santander/pipeline.yaml \
  --feature-groups configs/santander/feature_groups.yaml

# Resume from a specific stage (based on pipeline_state.json)
python -m core.pipeline.runner \
  --config configs/santander/pipeline.yaml \
  --resume
```

3-Stage Normalization (Must Understand)

Stage	Processing	Target
1	Power-law detection (skew+kurt -> log-log R^2) + log1p copy creation	High-skew columns
2	StandardScaler (TRAIN fit only)	Continuous columns (binary excluded)
3	Power-law _log copies are not scaled	Raw magnitude preserved

Note: Scaler must be fit on the TRAIN split only. val/test are transform-only.

Generator Input Routing

Generator inputs (GMM, TDA, etc.) are declared in `feature_groups.yaml`'s `input_filter`.

```
# feature_groups.yaml example
generator_params:
  input_filter:
    dtype: continuous      # continuous | all_numeric
    exclude_binary: true   # For GMM, TDA
    min_nunique: 20        # Exclude discrete variables
```

Prohibited: Hard-coded routing such as `product_cols`, `synth_cols` in adapters.

Phase 0 Output Verification (Pre-flight Check)

After Phase 0 completion, the following must be verified before training:

```
# 1. feature_stats.json -- check zero-variance, NaN ratio, feature count
python -c "
import json
stats = json.load(open('output/feature_stats.json'))
print(f'Feature count: {stats[\"num_features\"]}')
print(f'Zero-variance: {stats.get(\"zero_variance_cols\", [])}')
print(f'NaN ratio > 0.5: {[c for c,v in stats[\"nan_ratio\"].items() if v > 0.5]}')
"

# 2. label_stats.json -- check class balance, positive rate
python -c "
import json
stats = json.load(open('output/label_stats.json'))
for task, info in stats.items():
    print(f'{task}: {info}')
"

# 3. LeakageValidator (runs automatically, check logs)
# Automatically called before training in train.py -- auto-removes features with
correlation > 0.95
```

SageMaker Processing Job (Cloud)

```
# Run Phase 0 on SageMaker CPU instance
python scripts/submit_processing_job.py \
  --config configs/santander/pipeline.yaml \
  --instance-type ml.m5.2xlarge \
  --dry-run # Verify Job configuration first
```

Phase 1–3: Training + Ablation

Training Architecture Overview

PLE 2-Phase Training:

Phase 1 (15 epochs): Joint training -- all experts + task towers

Phase 2 (8 epochs): Tower fine-tune -- shared experts frozen, task heads only

Key components:

- 7 shared experts: deepfm, temporal_ensemble, hgc, perslay, causal, lightgcn, optimal_transport
- 1 task expert: mlp (per task)
- 12 tasks (4 tiers): 7 binary / 2 multiclass / 3 regression (18→13: 5 deterministic-leakage tasks removed → 13→12: segment_prediction removed 2026-05-01)
- Uncertainty weighting (Kendall et al.)
- AMP (Mixed Precision) must be enabled

Execution Modes

Mode 1: Local Direct Execution (Development/Debugging)

```
# Small-scale test (50K subsample, 3 epochs)
python containers/training/train.py \
  --config configs/santander/pipeline.yaml \
  --data-dir data/benchmark/ \
  --output-dir output/local_test/ \
  --epochs 3

# Full data local test (1 epoch end-to-end verification required)
python containers/training/train.py \
  --config configs/santander/pipeline.yaml \
  --data-dir data/benchmark/ \
  --output-dir output/full_test/ \
  --epochs 1
```

Mode 2: Docker Local Execution (Reproducing SageMaker Environment)

```
# Reproduce SageMaker environment identically with Docker
docker run --rm --gpus all \
  -v $(pwd)/data:/opt/ml/input/data/training \
  -v $(pwd)/output:/opt/ml/model \
  -v $(pwd)/configs:/opt/ml/input/config \
  ple-training:latest \
  --config /opt/ml/input/config/santander/pipeline.yaml
```

```
# Ablation scenario test
docker run --rm --gpus all \
  -v $(pwd)/data:/opt/ml/input/data/training \
  -v $(pwd)/output:/opt/ml/model \
  ple-training:latest \
  --removed-feature-groups "tda_global,gmm_clustering" \
  --epochs 3
```

Mode 3: SageMaker Cloud Execution

```
# Step 1: Build and stage source package to S3 (one-time; reuse across all Jobs)
python scripts/package_source.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml

# Step 2: Submit teacher training Job
python scripts/run_sagemaker_teacher.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --instance-type ml.g4dn.xlarge \
  --use-spot \
  --dry-run # Verify first

# Step 3: Submit evaluation Job
python scripts/run_sagemaker_eval.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --checkpoint s3://aiops-ple-financial/checkpoints/best/

# Step 4: Submit distillation Job
python scripts/run_sagemaker_distillation.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --teacher-checkpoint s3://aiops-ple-financial/checkpoints/best/
```

Ablation Execution (4-Dimension, 23 Scenarios)

Ablation Structure

Phase	Scenario Count	Content	Instance
0	1	Data Preparation	CPU (ml.m5.2xlarge)
1	16	Feature Group Ablation (bottom-up + top-down)	GPU
2	16	Expert Ablation (bottom-up + top-down)	GPU
3	16	Task x Structure Cross (4 tiers x 4 variants)	GPU
4	2	Best-Config Teacher + Distillation	GPU + CPU
5	1	Analysis + HTML Report	CPU

Local Ablation Execution

```
# Sequential execution on local GPU (RTX 4070, ~24 hours)
python scripts/run_local_ablation.py \
  --config configs/santander/pipeline.yaml \
  --phase 1 # Feature Group Ablation only

# Full Phase execution
python scripts/run_local_ablation.py \
  --config configs/santander/pipeline.yaml \
  --phase all
```

SageMaker Ablation Execution (Parallel)

```
# SageMaker Spot 4 instances parallel (~4 hours)
python scripts/run_santander_ablation.py \
  --config configs/santander/pipeline.yaml \
  --max-parallel 4 \
  --use-spot \
  --dry-run # Verify first

# Actual submission (budget guard auto-applied: $80 limit)
python scripts/run_santander_ablation.py \
  --config configs/santander/pipeline.yaml \
  --max-parallel 4 \
  --use-spot
```

Ablation Hyperparameters

The following HPs can be overridden per scenario:

Available feature groups for ablation (17 total; 2026-04-28 Santander v2):

- **Legacy groups (ablation baselines):** tda_global, tda_local, gmm_clustering, hmm_states, mamba_temporal
- **New groups (2026-04-25 addition):** txn_lag_tensor, txn_rolling_stats, nba_label_multihot, mcc_top30_multihot
- **Core groups (not recommended to remove):** demographics, product_holdings, txn_behavior

```
# Remove feature groups
--removed-feature-groups "tda_global,gmm_clustering"

# Remove experts
--removed-experts "hgcn,perslay"

# Adjust task count
--num-active-tasks 4 # 4/8/11/12

# Structural variants
--disable-adatt # Disable adaTT
--disable-ple-stacking # Disable PLE stacking (single CGC)
--ple-num-layers 2 # PLE depth: 1/2/3
```

```
# Loss weighting strategy
--loss-weighting-strategy uncertainty # uncertainty/gradnorm/dwa/fixed
```

Ablation Default Training Parameters

Read from pipeline.yaml's ablation.training_defaults:

```
ablation:
  training_defaults:
    epochs: 5
    batch_size: 6144
    learning_rate: 0.0005
    amp: true
    early_stopping_patience: 3
    seed: 42
    num_workers: 2
    pin_memory: true
    drop_last: true
```

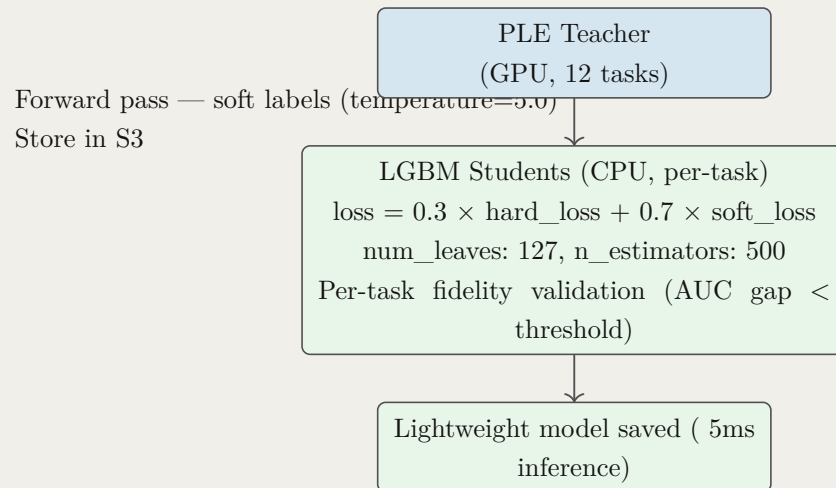


Figure 1: Teacher → LGBM Student knowledge distillation flow.

Phase 4: Knowledge Distillation

Teacher -> LGBM Student

Execution CLI

```

# Step 1: Generate teacher soft labels
python scripts/generate_soft_labels.py \
  --config configs/santander/pipeline.yaml \
  --checkpoint output/best_model/checkpoint.pt \
  --output output/soft_labels/

# Step 2: Train LGBM Student
python scripts/train_student.py \
  --config configs/santander/pipeline.yaml \
  --soft-labels output/soft_labels/ \
  --output output/student_models/

# Step 3: Fidelity Gate verification
python scripts/validate_fidelity.py \
  --teacher-metrics output/best_model/eval_metrics.json \
  --student-metrics output/student_models/eval_metrics.json
  
```

Distillation Settings (pipeline.yaml)

```

distillation:
  temperature: 5.0
  alpha: 0.3          # 30% hard + 70% soft
  lgbm:
  
```

```
num_leaves: 127
learning_rate: 0.05
n_estimators: 500
min_child_samples: 50
subsample: 0.8
colsample_bytree: 0.8
```

Fidelity Gate

Verifies that the distilled Student maintains a certain level of performance relative to the Teacher.

- AUC gap < threshold: Student model registered
- Failure: Alert, manual review
- Can be disabled during ablation with `--skip-fidelity-gate` flag

IG-Based Feature Selection (For Distillation)

Uses Integrated Gradients attribution from the Teacher model to select important features and reduce Student input dimensionality.

```
python scripts/run_ig_selection.py \
  --config configs/santander/pipeline.yaml \
  --checkpoint output/best_model/checkpoint.pt \
  --top-k 100 # Top 100 features
```

Phase 5: Serving

3-Tier Scaling Architecture

Tier	Configuration	Cost/month	Latency
Tier 1 (Small)	API Gateway -> Lambda Container Image + in-memory features	~\$0-1	~5ms
Tier 2 (Medium)	API Gateway -> Lambda Container Image + LanceDB (recommendation cases) + DynamoDB (L2a cache / audit)	~\$100-400	~10ms
Tier 3 (Large)	ALB -> ECS + Redis + RocksDB + LanceDB	~\$360	~5-8ms

Lambda Deployment

```
# Deploy LGBM Student model to Lambda
python scripts/deploy_lambda.py \
  --config configs/santander/pipeline.yaml \
  --model-path output/student_models/ \
  --memory-mb 1024 \
  --timeout 30

# Verify deployment
aws lambda invoke \
  --function-name ple-recommend \
  --payload '{"user_id": "test_001", "context": {}}' \
  response.json
```

Inference Flow

```
Request (user_id, context)
|
(1) Feature lookup (memory or LanceDB)    ~0.01ms or ~5ms
|
(2) Real-time context combination          ~0.1ms
|
(3) LGBM multi-task inference              ~5ms
|
(4) Output normalization + rationale gen. ~0.1ms
|
Total: ~5-10ms
```

Recommendation Rationale Generation

3-stage Interpretability Pipeline:

- **Stage A:** Integrated Gradients + Expert Redundancy CCA + CGC Gate Analysis
- **Stage B:** Multi Interpreter -> Template Reason Engine -> XAI Quality Evaluator
- **Stage C:** Context Vector Store (RAG) -> CPE Scoring -> Agentic Orchestrator

A/B Testing

```
# configs/serving/ab_test.yaml
ab_test:
  enabled: true
  variants:
    - name: control
      model: s3://bucket/models/lgbm-v1/
      weight: 90          # 90% traffic
    - name: treatment
      model: s3://bucket/models/lgbm-v2/
      weight: 10          # 10% traffic
  evaluation:
    primary_metric: click_through_rate
    min_sample_size: 10000
    significance_level: 0.05
  auto_promote:
    enabled: true
    min_improvement: 0.02  # Auto-switch on 2%+ improvement
```

Canary deployment: 5% -> 25% -> 50% -> 100% (immediate rollback on anomaly detection)

Config Guide

All parameters are read from YAML config. Hard-coding in Python code is prohibited.

Split-Config Architecture

Config is split into two layers:

- **configs/pipeline.yaml** (common): architecture, training HPs, experts, task structure, AWS defaults — shared across all datasets.
- **configs/datasets/santander.yaml** (dataset-specific): `id_col`, `date_col`, `label_cols`, `data_path`, dataset-specific overrides.

`config_builder.py` merges both files and is the **single source of truth** for constructing `PLEConfig`. All scripts (`train.py`, `run_sagemaker_*.py`, `run_local_ablation.py`) call `config_builder.build()` instead of reading YAML directly. This guarantees that any dataset-specific change never requires modifying `train.py` or ablation scripts.

```
# config_builder.py usage
from core.config_builder import build_config
config = build_config(
    pipeline_yaml="configs/pipeline.yaml",
    dataset_yaml="configs/datasets/santander.yaml",
    overrides={} # CLI overrides (ablation HP)
)
```

pipeline.yaml Key Settings

tasks Section

```
tasks:
  - name: churn_signal # Task name (unique); 12 tasks total: 7 binary, 2
    multiclass, 3 regression
    type: binary # binary | multiclass | regression
    loss: focal # focal | ce | huber
    loss_params:
      alpha: 0.85 # focal alpha (reflects positive rate ~5.1%)
      gamma: 2.0
    loss_weight: 2.0 # Uncertainty weighting initial weight
    label_col: churn_signal # Label column name
```

training Section

```
training:
  batch_size: 2048 # 941K / 2048 = ~460 steps/epoch
  epochs: 50
  learning_rate: 0.0005
```

```

weight_decay: 0.01
gradient_clip_norm: 5.0
seed: 42
phase1:
  epochs: 15
phase2:
  epochs: 8
  freeze_shared: true    # Freeze shared experts
early_stopping:
  patience: 7
  metric: val_loss
scheduler:
  type: cosine
  warmup_epochs: 3

```

data Section

```

data:
  id_col: customer_id
  backend: [cudf, duckdb, pandas] # Priority order
  temporal_split:
    enabled: true
    date_col: snapshot_date
    gap_days: 1                # Minimum 1-day gap (monthly snapshot)
    train_ratio: 0.7
    val_ratio: 0.15

```

aws Section

```

aws:
  region: ap-northeast-2
  s3_bucket: aiops-ple-financial
  instance_type: ml.g4dn.xlarge    # GPU (Phase 1-3)
  cpu_instance_type: ml.m5.2xlarge # CPU (Phase 0)
  use_spot: true
  max_run_seconds: 43200           # 12 hours

```

cold_start Section

Defines handling for customers with insufficient transaction history (cold start). Sequence-based features (HMM, Mamba, TDA local) are meaningless without history and are masked to 0, while demographic/product/global aggregate features are preserved.

```

cold_start:
  seq_col: txn_amount_seq          # Column for measuring history depth
  min_txn_count: 3                 # Below this -> cold start flag
  zero_features_prefix:           # Feature prefixes to mask to 0 on cold start
    - hmm_states
    - mamba_temporal
    - tda_local

```

```

keep_features_prefix:          # Features preserved even on cold start
(reference)
- prod_
- synth_
- tda_global
- gmm_
- graph_

```

Behavior:

1. If the `seq_col` column (LIST type) length is at or below `min_txn_count`, the customer is classified as cold start.
2. Generated features matching `zero_features_prefix` are replaced with 0 — these features produce only noise without sufficient sequences.
3. Features matching `keep_features_prefix` retain their original values regardless of cold start status.

feature_groups.yaml Core Structure

5-axis feature architecture:

Axis	Description	Example Group
State	Static demographic/account attributes	demographics, product_holdings
Snapshot	Long-term sequence summaries	TDA global, HMM, trends
Timeseries	Short-term time-series patterns	TDA local, Mamba, txn
Hierarchy	Product/MCC tree embeddings	Poincare embeddings
Item	Collaborative filtering	LightGCN, product holdings

Key fields per group:

```

- name: demographics          # Group name
  group_type: transform       # transform | generator
  columns: [age, income]     # Input columns
  output_dim: 11             # Output dimension (5 numeric + 6 categorical
LabelEncoded)
  target_experts: [deepfm, mlp] # Experts to route to
  distill: true              # Whether to include in distillation

```

task_groups Section (GradSurgery / task-type projection experiment)

Note: adaTT degrades at 13-task scale. GradSurgery (PCGrad task-type projection) was tested as an alternative but was not adopted — it showed no meaningful improvement over the PLE-only baseline and introduced VRAM overhead. The production configuration disables both adaTT and GradSurgery.

```

# grad_surgery section (tested as adaTT task_groups alternative; not adopted in
production)
grad_surgery:

```

```
enabled: true
task_type_groups:
  binary: [churn_signal, will_acquire_deposits, will_acquire_investments,
           will_acquire_accounts, will_acquire_lending, will_acquire_payments,
           top_mcc_shift]
  multiclass: [nba_primary, next_mcc]
  regression: [product_stability, cross_sell_count, mcc_diversity_trend]
```

task_relationships (Logit Transfer)

Three transfer methods:

Method	Description
output_concat	Concatenate source task output to target tower input
hidden_concat	Share source task hidden representation
residual	Add source task output via residual connection

Monitoring

Data Drift

```
# configs/monitoring.yaml
monitoring:
  drift:
    metric: psi          # Population Stability Index
    warning_threshold: 0.1
    critical_threshold: 0.25
    check_frequency: daily
    action_on_critical: trigger_retraining # After 3 consecutive days
```

```
# Manual drift check
python scripts/check_drift.py \
  --reference data/benchmark/benchmark_v2.parquet \
  --current data/latest/latest.parquet \
  --threshold 0.1
```

Model Performance Monitoring

```
monitoring:
  model:
    metrics: [auc, mae, fl_macro]
    degradation_threshold: 0.05 # 5%+ performance degradation
    action: notify              # notify | auto_retrain
```

Fairness Audit

Verifies model prediction fairness. Monitors performance differences by segment/gender/age_group.

Lineage Tracking

3-tier audit architecture:

- **Layer 1** (automatic): CloudTrail, S3 Versioning, SageMaker Lineage
- **Layer 2** (semi-automatic): ExperimentTracker, SchemaRegistry, ModelRegistry
- **Layer 3** (explicit): AuditLogger, ComplianceChecker, AccessController

```
# Query lineage
aws s3 ls s3://aiops-ple-financial/audit/lineage/ --recursive

# Query SageMaker Experiments
python -c "
from sagemaker.analytics import ExperimentAnalytics
```

```
analytics = ExperimentAnalytics(experiment_name='santander_ple')
df = analytics.dataframe()
print(df[['TrialName', 'auc_roc', 'val_loss']].head(20))
"
```

Champion/Challenger Evaluation (Offline Gate)

`scripts/submit_pipeline.py::_decide_promotion` auto-decides in this order, and every outcome is written by `AuditLogger.log_model_promotion` (HMAC + hash chain) to the S3 WORM audit log.

```
(1) --force-promote flag present?    -> promote (operator override)
(2) No current champion?             -> promote (bootstrap)
(3) fidelity_summary.failed > 0?     -> reject (safety floor)
(4) ModelCompetition.evaluate():
    - primary metric improves >= 0.5%
    - secondary metric degrades <= 2%
    - (optional) paired bootstrap significance
    approved                        -> promote
    rejected                        -> register only (promoted=False)
```

Condition	Result	audit decision	-- --	--force-promote	Always promote (manual override)
<code>force_promote</code>		No champion		Bootstrap promote	<code>bootstrap</code>
fidelity failures ≥ 1		Register only (safety floor)		reject	
Competition approves		Auto-promote		<code>promote</code>	
Competition rejects		Register only (no promotion)		reject	

The online gate (`ModelMonitor.evaluate_champion_challenger`, two-proportion z-test on DynamoDB prediction logs) is reserved for post-accumulation evaluation and is scheduler-triggered separately.

Troubleshooting (FAQ)

NaN Loss During Phase 2 Training

Cause: Double-sigmoid issue where sigmoid-applied values are fed into FocalLoss again, or NaN in labels.

Solution:

```
# 1. Pass pre-activation logits to FocalLoss (prevent sigmoid duplication)
# 2. Check which task produced NaN in logs
grep "NaN.*loss" output/training.log

# 3. Check label NaN ratio
python -c "
import pyarrow.parquet as pq
labels = pq.read_table('output/labels.parquet')
for col in labels.column_names:
    null_count = labels.column(col).null_count
    print(f'{col}: {null_count}/{len(labels)} ({null_count/len(labels)*100:.1f}%)')
"
```

Low GPU Utilization (< 50%)

Cause: DataLoader bottleneck, small batch_size, or CPU preprocessing bottleneck.

Solution:

```
# 1. Increase batch_size (within VRAM limits)
# 941K data -> batch_size: 5632 recommended (validated)

# 2. Check DataLoader num_workers
# pipeline.yaml ablation.training_defaults.num_workers: 2

# 3. Verify pin_memory is enabled
# pipeline.yaml ablation.training_defaults.pin_memory: true

# 4. Check GPU memory usage (logged every epoch)
grep "gpu_memory" output/training.log
```

Suspected Label Leakage

Cause: Last timestep of sequence overlaps with label, or scaler was fit on val/test data.

Solution:

```
# 1. Check LeakageValidator logs
grep "LeakageValidator" output/training.log
```

```
grep "correlation.*>.*0.95" output/training.log

# 2. Verify sequence truncation settings
# pipeline.yaml sequences.product_sequences.truncate_last: 1
# -> Removes month 17 (label month), uses only months 1-16

# 3. Verify prod_* columns are recalculated from seq_* month 16
# pipeline.yaml data.preprocessing.leakage_prevention.recompute_prod_from_seq: true
```

SageMaker Job Timeout

Cause: Insufficient `max_run_seconds` or overestimated data size.

Solution:

```
# 1. Check max_run_seconds
# pipeline.yaml aws.max_run_seconds: 43200 (12 hours)

# 2. Estimate time from small-scale test
# 50K x 3 epochs -> measured time x (941K/50K) x (full_epochs/3)

# 3. Distinguish Spot interruption from algorithm error
# Check SageMaker SecondaryStatusTransitions
aws sagemaker describe-training-job \
  --training-job-name <job-name> \
  --query 'SecondaryStatusTransitions'
```

Frequent Spot Instance Interruptions

Cause: Exceeding 4 concurrent instances causes AZ competition and sharply increased interruption frequency.

Solution:

```
# 1. Limit concurrent Spot instances to 4 or fewer
# scripts/run_santander_ablation.py --max-parallel 4

# 2. Set max_wait: max_run + 1 hour
# 10-hour wait is wasteful -- max_wait = max_run_seconds + 3600

# 3. Check checkpoints (saved to S3 every epoch)
aws s3 ls s3://aiops-ple-financial/checkpoints/<job-name>/
```

Gradient Norm Explosion

Cause: Excessive learning rate or data outliers.

Solution:

```
# Check gradient_clip_norm (pipeline.yaml training.gradient_clip_norm: 5.0)
# Check warning logs when exceeding 10x the clip threshold
grep "gradient.*norm.*exceed" output/training.log

# Reduce learning_rate: 0.0005 -> 0.0001
```

Cost Management

Spot Instance Strategy

Item	On-Demand	Spot
g4dn.xlarge per hour	\$0.526	~\$0.16 (70% savings)
50 epochs (~4 hours)	\$2.10	~\$0.64
23 ablation scenarios (14 joint feature+expert + 9 structure cross)	~\$100	~\$30

Cost Check CLI

```
# Check current month cost (required before SageMaker submission)
aws ce get-cost-and-usage \
  --time-period Start=$(date -d "$(date +%Y-%m-01)" +%Y-%m-%d),End=$(date +%Y-%m-%d) \
  --granularity MONTHLY \
  --metrics BlendedCost \
  --filter '{"Dimensions":{"Key":"SERVICE","Values":["Amazon SageMaker"]}}'
```

Budget Guard

When pipeline.yaml's ablation.budget_limit: 80.0 (USD) is exceeded, ablation automatically stops.

```
# Check budget setting
grep "budget_limit" configs/santander/pipeline.yaml
# ablation.budget_limit: 80.0
```

Cost Optimization Checklist

1. **Disable ProfilerReport:** disable_profiler=True (SageMaker estimator)
2. **Enable AMP:** ~2x speedup on g4dn T4 GPU
3. **Optimize batch_size:** 941K -> 5632 recommended (AMP FP16, lr 0.0005)
4. **Spot concurrency <= 4:** Prevent AZ competition
5. **max_wait = max_run + 1 hour:** Prevent excessive waiting
6. **Build source package once:** Reuse across all Jobs
7. **Phase 0 on CPU instances:** Prevent GPU waste
8. **Enable Warm Pool:** Reuse instances across consecutive Jobs
9. **Check S3 result existence:** Prevent duplicate Jobs
10. **Verify actual cost post-experiment:** Investigate if 2x+ deviation from estimate

Infrastructure Cost Comparison

Approach	Monthly Cost (Est.)	Notes
Self-built K8s (4 GPUs)	\$8,000–15,000	Hardware depreciation + personnel
SageMaker On-Demand	\$500–2,000	Weekly training + serving
SageMaker Spot + Lambda	\$200–800	When optimized

Orchestration Cost

Airflow (existing): 4 containers always-on -> ~\$200-300/month
Step Functions (AWS): 5 state machines x 2/week -> ~\$0.002/month (effectively free)

Appendix: Operations Rules Summary

Absolute Prohibitions

- No code debugging on SageMaker (USD 0.50+ per submission)
- No preprocessing code in `train.py`
- No hard-coded generator calls in Adapter
- No hard-coded mappings like `FEATURE_GROUP_COLUMN_PREFIXES`
- Avoid direct pandas usage (cuDF -> DuckDB -> pandas fallback order)
- No proceeding to next Phase without verifying experiment results

Separation of Concerns Principle

Module	Role	Must NOT Do
Adapter	raw data -> standardized DataFrame	Preprocessing, feature generation, label derivation
PipelineRunner	Preprocessing -> feature gen -> normalization -> tensor save	Model training
<code>train.py</code>	Data load -> model build -> training	Preprocessing (fillna, scaler, etc.)
Ablation script	Scenario orchestration	Hard-coded scenario/expert lists

Development Order (Must Follow)

1. Local 50K subsample test (verify end-to-end success)
2. Local full data 1-epoch test
3. Docker environment reproduction test
4. SageMaker `--dry-run` verification
5. SageMaker submission

Checkpoint Resume

Training checkpoints are saved to S3 on each epoch. The trainer supports both `.pt` and `.pth` file patterns when scanning for the latest checkpoint (file pattern fix applied — both extensions are checked).

Epoch counting: After resume, epoch numbering continues from the saved epoch index (not reset to 0). The `trainer.py` reads `checkpoint["epoch"]` and sets `start_epoch = checkpoint["epoch"] + 1`.

```
# Resume training from latest checkpoint in S3
python scripts/run_sagemaker_teacher.py \
  --config configs/pipeline.yaml \
  --dataset configs/datasets/santander.yaml \
  --resume # auto-detects latest .pt/.pth in output S3 path
```

Pipeline State Auto-Recovery

pipeline_state.json tracks completed stages. On interruption, resume with --resume flag.

```
{
  "completed_stages": ["adapter", "temporal_prep", "schema",
    "encryption", "features", "labels", "leakage",
    "sequences", "dataloader", "training"],
  "artifacts": {
    "features": {"path": "features.parquet", "rows": 941132, "dim": 512}
  }
}
```

Ops/Audit Agent Integration

Each pipeline stage completion triggers an event to OpsAgent via `_PipelineState.mark_complete()` callbacks. OpsAgent inspects 7 checkpoints (CP1 Ingestion through CP7 A/B Testing), while AuditAgent audits reason quality (3-Tier verification) and regulatory compliance.

Changes (code, configuration, model, data) are detected via dual push/pull channels, automatically re-executing checklists for affected pipeline parts.

Detailed design: Design Document 11 ([docs/design/11_ops_audit_agent.typ](#))